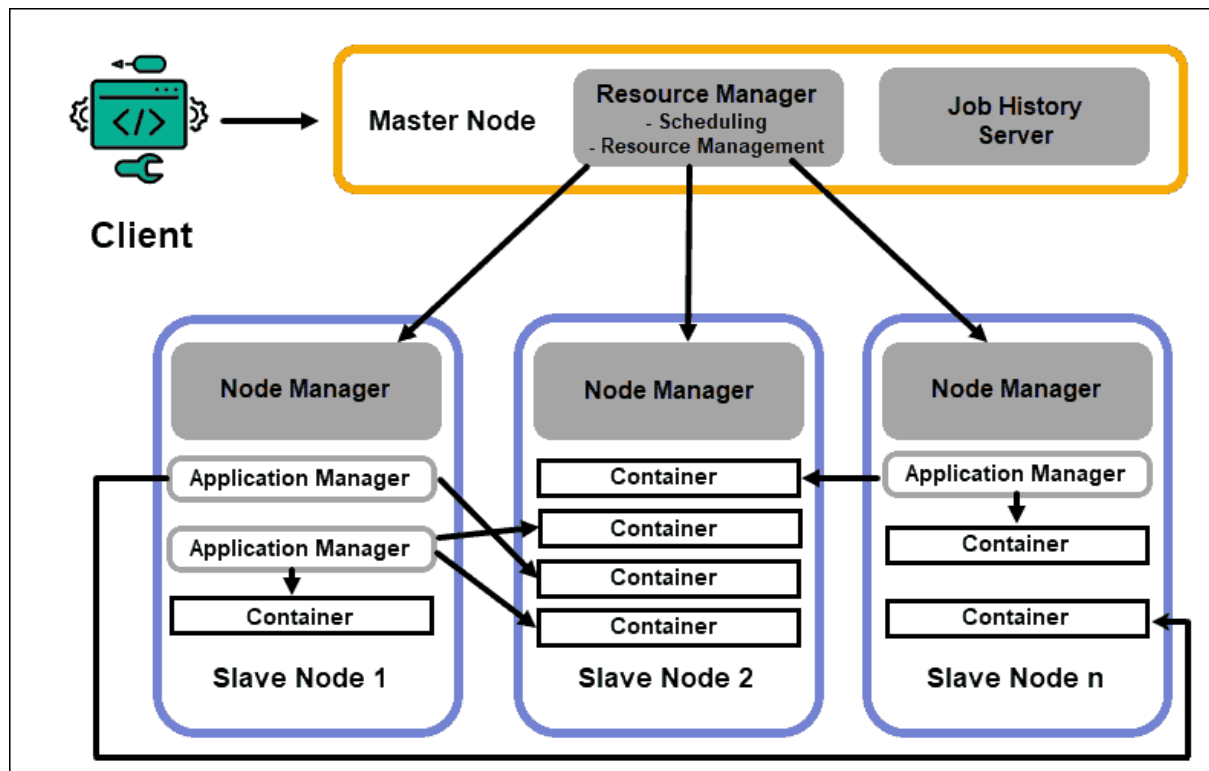


Apache Hadoop and Apache Spark are two open-source frameworks you can use to manage and process large volumes of data for analytical purposes. Organizations need to process data at scale and speed to obtain real-time insights for business intelligence. Apache Hadoop allows you to cluster multiple computers to analyze massive datasets in parallel faster. Apache Spark uses in-memory caching and optimized query execution to help quickly analyze data of any size. Spark is a more advanced technology than Hadoop because it utilizes artificial intelligence and machine learning(AI/ML) in the data processing process. However, many companies use a combination of Spark and Hadoop to meet their data analytics goals. What are the similarities between Hadoop and Spark?



Both Hadoop and Spark are distributed systems that allow you to process data at scale. They are capable of recovering from failures if data processing is interrupted for any reason.

Distributed Big Data Processing

Big data is collected regularly, continuously, and on a large scale in many different formats.

To store, manage, and process large datasets, Apache Hadoop divides datasets into smaller subsets or partitions. Apache Hadoop then stores these partitions on a

distributed network of servers. Similarly, Apache Spark processes and analyzes large datasets across distributed nodes to provide business insights.

Depending on the use case, you may need to integrate both Hadoop and Spark with different software to optimize functionality.

Fault tolerance

Apache Hadoop continues to run even if one or more nodes processing data fail. Apache Hadoop creates multiple copies of the same data block and stores them on several nodes. When a node fails, Hadoop retrieves information from another node and prepares the information for data processing.

Meanwhile, Apache Spark relies on a special data processing technology called Flexible distributed dataset (RDD). Thanks to RDDs, Apache Spark remembers how to retrieve specific information from the repository and can recreate the data if the underlying repository fails.

Feature	APACHE HADOOP	APACHE SPARK
Data Processing	MapReduce (Disk-based)	In-memory processing (RAM-based)
Speed	Slower due to disk I/O in MapReduce	Faster due to in-memory processing
Ease of Use	Complex, requires writing MapReduce jobs	Easier to use, supports high-level APIs
Real-time Processing	Not suitable for real-time processing	Supports real-time processing with Spark Streaming
Fault Tolerance	Built-in with HDFS replication	Built-in with RDD (Resilient Distributed Datasets)
Data Processing Model	Batch processing	Batch and stream processing
Language Support	Java (MapReduce), other languages with additional libraries	Scala, Java, Python, R
Cost Efficiency	Cost-effective on commodity hardware	More memory-intensive, can be costlier
Use Case	Best for large-scale batch processing	Ideal for both batch and real-time analytics

The main components of the Hadoop and Spark frameworks.

Both Hadoop and Spark are made up of many interacting and collaborative software modules that make the system work.

Components of Hadoop

Apache Hadoop has four main components:

- Hadoop Distributed File System (HDFS) is a specialized file system that stores large datasets across multiple computers. These computers are called...Hadoop cluster.
- Yet Another Resource Negotiator (YARN) schedules tasks and allocates resources to applications running on Hadoop.
- Hadoop MapReduce allows programs to break down large data processing tasks into smaller tasks and run them in parallel across multiple servers.

- Hadoop Common, or Hadoop Core, provides the necessary software libraries for other Hadoop components.

Spark components

Apache Spark runs with the following components:

- Spark Core orchestrates the fundamental functions of Apache Spark. These functions include memory management, data storage, task scheduling, and data processing.
- Spark SQL allows you to process data in Spark's distributed storage.
- Spark streaming and structured streaming enable Spark to efficiently transmit data in real time by breaking the data into small, continuous blocks.
- The Machine Learning Library (MLlib) provides several machine learning algorithms that you can apply to big data.
- GraphX allows you to visualize and analyze data using graphs.

The main difference between Hadoop and Spark

Both Hadoop and Spark allow you to process big data in different ways.

Apache Hadoop was created to delegate data processing to multiple servers instead of running workloads on a single machine.

Meanwhile, Apache Spark is a newer data processing system that overcomes the main limitations of Hadoop. While capable of handling large datasets, Hadoop only processes in batches and has significant latency.

Architecture

Hadoop has a native file system called the Hadoop Distributed File System (HDFS). HDFS allows Hadoop to divide large data blocks into many smaller, homogeneous blocks. HDFS then stores these smaller data blocks across multiple groups of servers.

Meanwhile, Apache Spark does not have its own native file system. Many organizations run Spark on Hadoop's file system to store, manage, and retrieve data.

Or you can also use Amazon Redshift or Amazon Simple Storage Service (Amazon S3) Make data storage options available for Spark.

Performance

Hadoop can process large datasets in batches, but it can be slower. To process data, Hadoop reads information from external storage and then analyzes and feeds the data into the software's algorithms.

For each data processing step, Hadoop writes the data back to an external storage, thereby increasing latency. Therefore, Hadoop is not suitable for real-time processing tasks but is an ideal choice for workloads with acceptable latency. For example, Hadoop is suitable for analyzing monthly sales records. However, Hadoop may not be the best choice for determining real-time brand sentiment from social media feeds.

On the other hand, Apache Spark is designed to handle massive amounts of data in real time.

Instead of accessing data from external storage, Spark copies the data to RAM before processing it. Spark only writes the data back to external storage after a specific task is complete. Writing and reading from RAM is many times faster than the same process on an external drive. Furthermore, Spark reuses retrieved data for multiple operations. Therefore, Spark performs better than Hadoop to varying degrees for both simple and complex data processing.

Machine learning

Apache Spark provides a machine learning library called MLlib. Data scientists use MLlib to run regression analysis, classification, and other machine learning tasks. You can also train machine learning models with both unstructured and structured data and deploy them to business applications.

In contrast, Apache Hadoop doesn't have built-in machine learning libraries. Instead, you can integrate Spark with other software like Apache Mahout to build machine learning systems. The choice of software depends on the specific requirements of the workload. You can consider factors such as the size and complexity of the data, the type of machine learning model you want to use, as well as performance requirements and the scalability of the application.

Security

Apache Hadoop is designed with robust security features to protect data. For example, Hadoop uses encryption and access control to prevent unauthorized parties from accessing and manipulating the data repository.

However, Apache Spark itself has limited security capabilities. According to the Apache Software Foundation, you must enable Spark's security features and ensure that the environment in which Spark runs is secure.

Spark supports various deployment methods, each with different levels of security. For example, deploying Spark on Hadoop improves overall security thanks to Hadoop's encrypted distributed storage.

Scalability

Scaling is easier on Hadoop than on Spark. If you need more processing power, you can add nodes or computers on Hadoop at a reasonable cost.

Conversely, scaling up for a Spark deployment typically requires investing in more RAM. Costs can add up rapidly for on-premises infrastructure.

Expense

Apache Hadoop is more affordable to set up and run because the framework uses hard drives for data storage and processing. You can set up Hadoop on standard or low-configuration computers.

Meanwhile, Spark is more expensive to handle large amounts of data because the framework uses RAM for in-memory processing. RAM is typically more expensive than a hard drive with the same storage capacity.

Comparing when to use Hadoop or Spark

Apache Spark was created to overcome the limitations of Hadoop's external storage access architecture. Apache Spark replaces Hadoop's native data analysis library, MapReduce, with faster machine learning processing capabilities.

However, Spark and Hadoop are not incompatible. While Apache Spark can run as a standalone framework, many organizations use both Hadoop and Spark for big data analytics.

Depending on your specific business requirements, you can use Hadoop, Spark, or both to process data. Here are some things to consider when making your decision.

Adjusting the scale of cost savings

Apache Hadoop is a better choice for building and scaling a cost-effective data processing workflow. Adding more computers to an existing Hadoop cluster can

increase Hadoop's processing capacity. Doing so is more cost-effective than buying additional RAM to scale the Apache Spark framework.

Batch data processing

Batch processing is when you process large amounts of data without being limited by a predetermined timeframe. When prioritizing batch processing, organizations will use Apache Hadoop because this framework supports parallel processing across multiple nodes. For example, you can use Hadoop to generate inventory reports with no up-to-date requests from tens of thousands of records.

Real-time analysis

Use Apache Spark if you need to handle rapidly changing data. A data stream is information or data that is continuously transmitted by software. Apache Spark can handle live data streams and provide in-depth analysis in real time. For example, financial institutions use Apache Spark to detect fraud in ongoing transactions and alert bank employees.

Machine learning features

Machine learning involves training software functions or models using large datasets. Apache Spark is better suited for such tasks thanks to its built-in machine learning library. This means Spark can train machine learning models in real time without additional integration.

Security, speed, and interactive analytics.

You can use Hadoop and Spark to benefit from the strengths of both frameworks. Hadoop offers secure distributed processing at an affordable cost. If you run Spark on Hadoop, you can migrate time-sensitive workloads, such as graph analysis tasks, to Spark's in-memory data processing. You get performance and secure processing on external storage for your analytics.